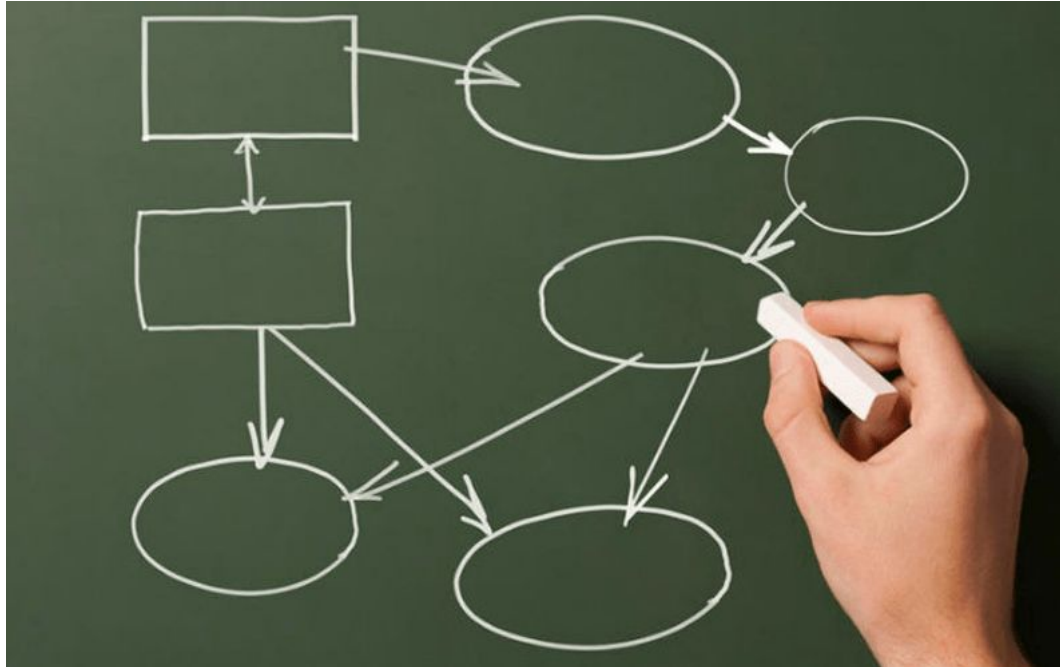
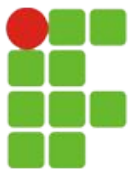
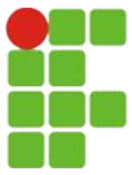




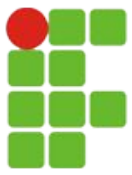
Algoritmos e programação– ALG

Prof. Jones



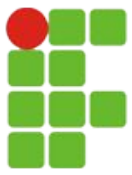
Últimas aulas ...

- ✓ **Funções**
- ✓ **Procedimentos**
- ✓ **STL ☐ bits/stdc++.h (Containers)**
 - ✓ **Pair**
 - ✓ **Tuple**



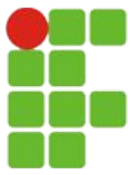
Vetores

ARRAYS



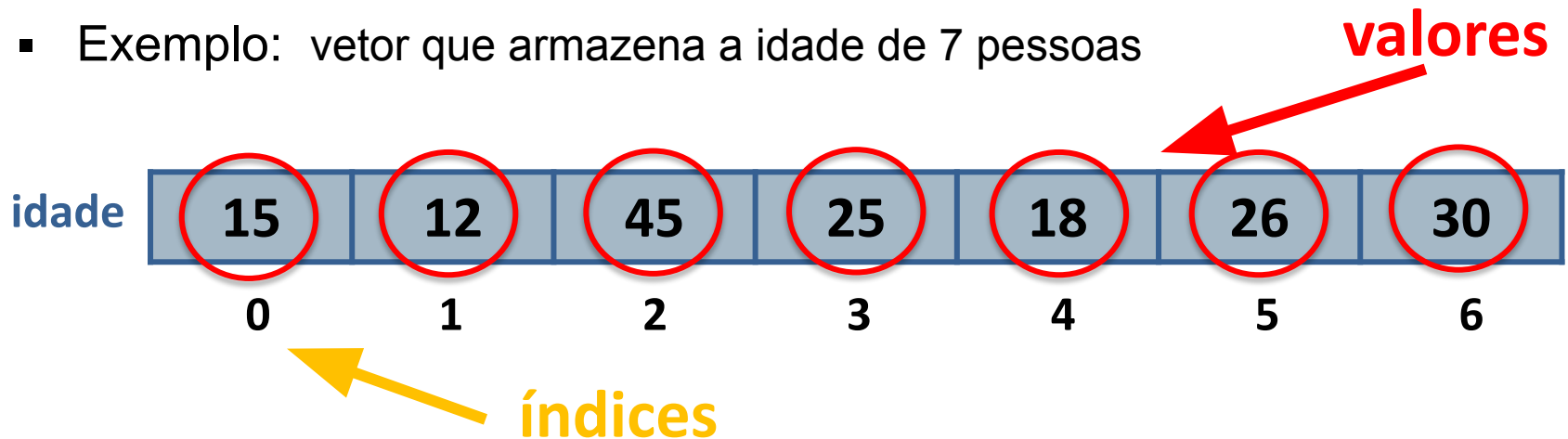
Conceito

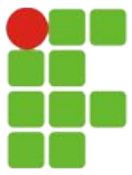
- ✓ Vetor (**array unidimensional**) é uma estrutura de dados linear que armazena uma lista de valores de mesmo tipo, que podem ser acessados individualmente a partir de um único identificador (índice).
- ✓ É uma variável composta **homogênea unidimensional**:
- ✓ Pode armazenar vários valores (composta);
- ✓ Todos os valores são de mesmo tipo de dados(homogênea);
- ✓ Possui uma única dimensão (unidimensional)



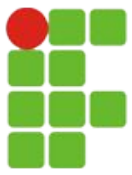
Estrutura de um vetor

- Para selecionar um elemento deste conjunto, fazemos o uso de um índice
 - é um número inteiro ou uma variável inteira, que indica a posição de um elemento no vetor;
- Os elementos de um vetor são armazenados em **posições contíguas da memória**.
- Exemplo: vetor que armazena a idade de 7 pessoas





Declaração

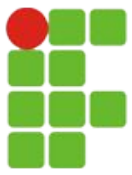


Declaração

- Sintaxe:

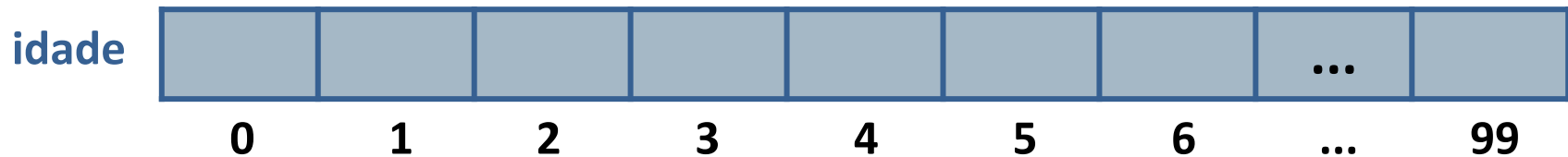
`<tipo de dado> <identificador> [<quantidade máxima de elementos>];`

- **tipo de dado:** `int, char, float, double;`
- **identificador:** mesmas regras de nomes de variáveis;
- **quantidade máxima de elementos:** valor constante que indica quantos elementos o vetor pode ter.
- Exemplo: `int idade[100];`

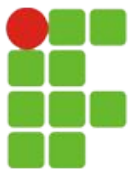


Declaração

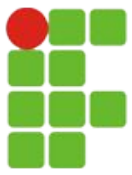
- Exemplo: `int idade[100];`



- cada uma das 100 posições pode armazenar um valor do tipo `int`;
- Cada posição tem um índice, de zero até 99 (tamanho menos 1)



Acesso

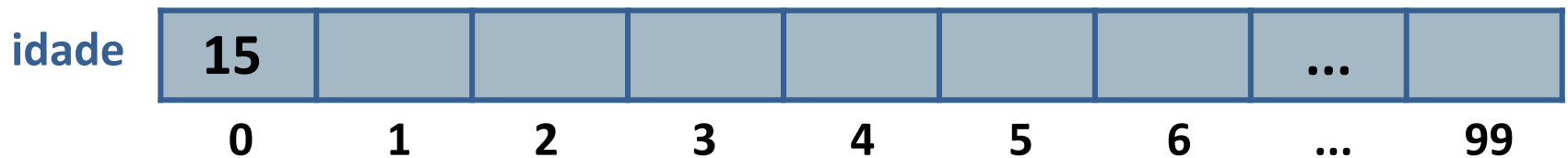


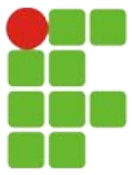
Acesso ao dado no vetor

- O acesso a cada elemento do vetor é feito através do índice colocado entre colchetes:
- Exemplos:

```
idade[0] = 15;
```

**atribuição da idade 15 ao
primeiro elemento do vetor**



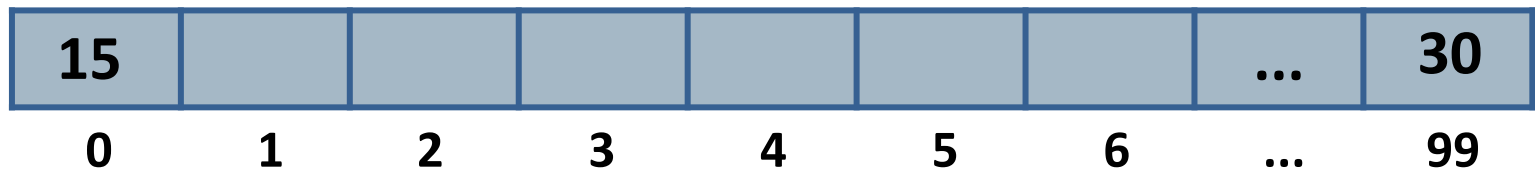


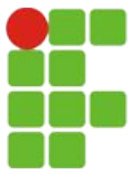
Exemplo

- Exemplos: `idade[99] = 2 * idade[0];`

**atribuir para a última posição
do vetor, o dobro do valor do
primeiro elemento**

idade

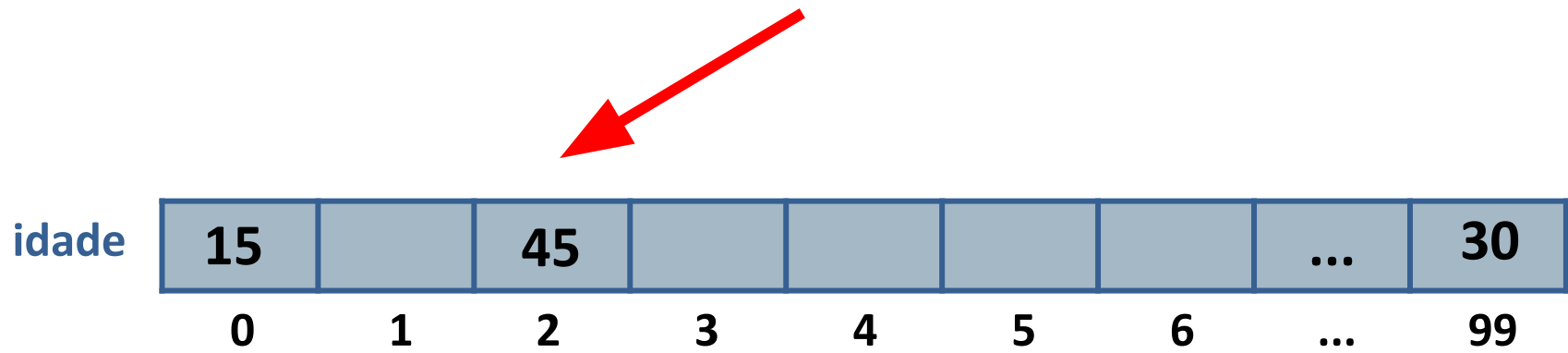


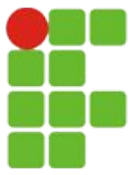


Exemplo

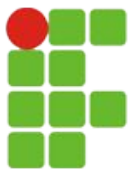
- Exemplos: `idade[2] = idade[0] + idade[99];`

**atribuir ao terceiro elemento, a
soma do valor do primeiro com o
último elemento**





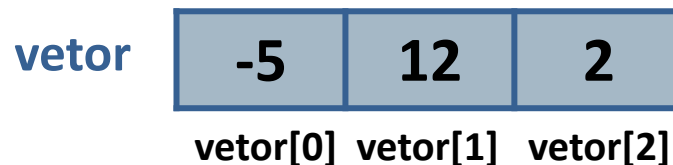
Inicialização

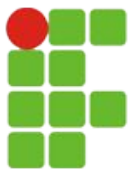


Inicialização

- Vetores quando criados também contém os “lixos de memória” em cada uma de suas posições;
- Eles podem ser inicializados no momento da declaração:

```
int vetor[3] = {-5, 12, 2};
```





Inicialização

- Caso não sejam inicializados todos os elementos do vetor, estes que não foram inicializados receberão o valor **zero**

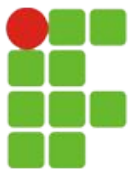
```
int vetor[6] = {-5, 12, 2};
```

vetor

-5	12	2	0	0	0
----	----	---	---	---	---

vetor[0] vetor[1] vetor[2] vetor[3] vetor[4] vetor[5]

```
vetor[0] = -5  
vetor[1] = 12  
vetor[2] = 2  
vetor[3] = 0  
vetor[4] = 0  
vetor[5] = 0
```



Inicialização

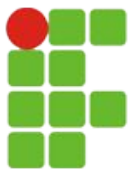
- É possível declarar o vetor **sem especificar a quantidade de posições**, apenas atribuindo os valores:

```
int vetor[] = {-5, 12, 2};
```

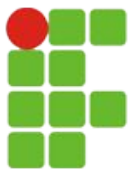
- O compilador calculará o número de elementos para armazenar:

o vetor possuirá **3** posições

vetor	-5	12	2
	vetor[0]	vetor[1]	vetor[2]



Exemplo prático



- Problema URI – 1214 – Acima da Média:

1.v. exemplo

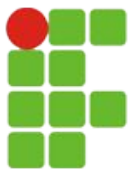
Entrada

A entrada contém muitos casos de teste. A primeira linha da entrada contém um inteiro C , indicando o número de casos de teste. Seguem C casos de teste ou instâncias. Cada caso de teste inicia com um inteiro N , que é o número de pessoas de uma turma ($1 \leq N \leq 1000$). Seguem N inteiros, separados por espaços, cada um indicando a média final (um inteiro entre 0 e 100) de cada um dos estudantes desta turma.

Saída

Para cada caso de teste imprima uma linha dando o percentual de estudantes que estão acima da média da turma com o valor arredondado e com 3 casas decimais.

Exemplo de Entrada	Exemplo de Saída
5	40.000%
5 50 50 70 80 100	57.143%
7 100 95 90 80 70 60 50	33.333%
3 70 90 80	66.667%
3 70 90 81	55.556%
9 100 99 98 97 96 95 94 93 91	

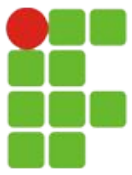


1.v. exemplo

- Problema URI – 1214 – Acima de Média:

Estudo do enunciado:

- Entradas: **C**: número de casos de teste
n: número de pessoas
nota: **vetor** com **nota** de cada aluno
- Saídas: **percentual** de estudantes acima da média da turma

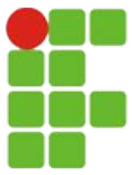


- Problema URI – 1214 – Acima de Média:

1.v. exemplo

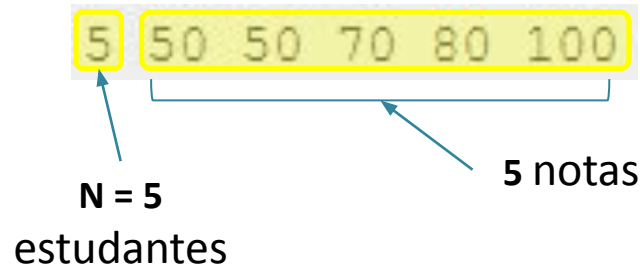
Estudo do enunciado:

- Operações: iterar casos de teste: $0 \leq i < c$
 - para cada caso de teste:
 - ler n
 - iterar pessoas: $0 < j \leq n$
 - ler $\text{nota}[j]$
 - somar: $\text{soma} = \text{soma} + \text{nota}[j]$
 - calcular média da turma: $\text{média} = \text{soma}/n$
 - iterar pessoas: $0 \leq j < n$
 - verificar se $\text{nota}[j]$ é maior que a **média**
 - contar**: $\text{cont} = \text{cont} + 1$
 - calcular **percentual** = $\text{cont} / n * 100$



- Problema URI – 1214 – Acima de Média:

1.v. **exemplo**



1. Calcular a média da turma:

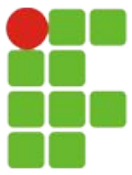
$$media = \frac{50 + 50 + 70 + 80 + 100}{5}$$

2. Determinar número de estudantes acima da média: **2**



3. Calcular o percentual:

$$percentual = \frac{2}{5} = 0,4 = 40\%$$



- Problema URI – 1214 – Acima de Média:

1.v. exemplo

```
#include <bits/stdc++.h>

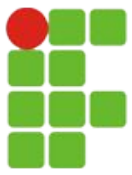
using namespace std;

int main()
{
    int c,i,n,j, mf[1000], contM;
    double soma, media, porcentagem;
    cin >> c;
    for(i = 0; i < c; i++)
    {
        cin >> n;
        soma = 0;
        for(j = 0; j < n; j++)
        {
            cin >> mf[j];
            soma += mf[j];
        }
        media = soma/n;
        contM = 0;
        for( j = 0; j < n; j++)
        {
            if(mf[j] > media)
            {
                contM++;
            }
        }
        porcentagem = (double)(100*contM)/n;
        cout << fixed << setprecision(3) << porcentagem << "%\n" << endl;
    }
    return 0;
}
```

declaração do vetor → `mf[1000]`

leitura dos elementos do vetor → `cin >> mf[j];`

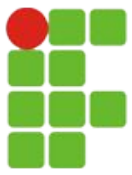
acesso a cada elemento do vetor → `mf[j]`



STL

include <bits/stdc++.h>

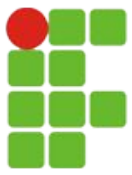
MAP



MAP

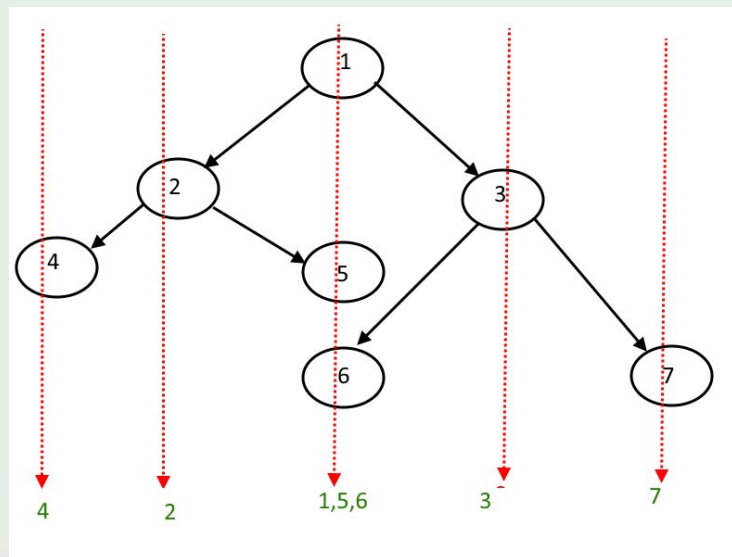
São contêineres associativos que armazenam elementos de forma mapeada. Cada elemento possui um **valor-chave** e um valor **mapeado**.

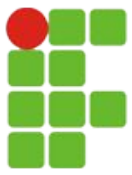
O MAP generaliza os índices dos vetores com qualquer tipo primitivo. Os índices podem ser “strings”, “booleanos”, “floats”, “chars” etc....



Pontos importantes:

- Facilita a implementação de determinados problemas;
- É uma **estrutura ordenada**, assim como o set
- Método baseado em árvore de busca





MAP

Sintaxe de implementação:

```
map <tipo1, tipo2> nome_da_variável;
```



CHAVE



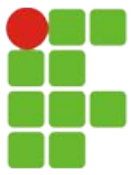
CONTEÚDO



EXEMPLO: Lista de DDDs do Brasil

Para consultar a lista de DDDs de cada macro região é só clicar em cima do DDD que vai abrir a lista de cidades daquele DDD.

- [DDD 11 – São Paulo – SP](#)
- [DDD 12 – São José dos Campos – SP](#)
- [DDD 13 – Santos – SP](#)
- [DDD 14 – Bauru – SP](#)
- [DDD 15 – Sorocaba – SP](#)
- [DDD 16 – Ribeirão Preto – SP](#)
- [DDD 17 – São José do Rio Preto – SP](#)
- [DDD 18 – Presidente Prudente – SP](#)
- [DDD 19 – Campinas – SP](#)
- [DDD 21 – Rio de Janeiro – RJ](#)
- [DDD 22 – Campos dos Goytacazes – RJ](#)
- [DDD 24 – Volta Redonda – RJ](#)
- [DDD 27 – Vila Velha/Vitória – ES](#)
- [DDD 28 – Cachoeiro de Itapemirim – ES](#)
- [DDD 31 – Belo Horizonte – MG](#)
- [DDD 32 – Juiz de Fora – MG](#)
- [DDD 33 – Governador Valadares – MG](#)
- [DDD 34 – Uberlândia – MG](#)

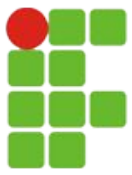


EXEMPLO:

```
#include<bits/stdc++.h>

using namespace std;

int main()
{
    map<int, string> mp;
    mp[11] = "São Paulo";
    mp[17] = "Barretos";
    mp[34] = "Planura";
    cout << mp[17] << endl;
    return 0;
}
```

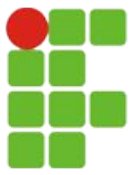


Como exibir todos elementos de um MAP?

O MAP é uma derivação do pair com o set.

Para percorrer um MAP usamos as funções do pair

```
for(auto i : mp) {  
    cout << i.first << " " << i.second << endl;  
}
```



EXEMPLO 2:

Queremos mapear as idades dos alunos do 1º INFO

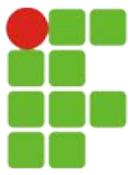
```
#include<bits/stdc++.h>

using namespace std;

int main()
{
    map<string,int> idade;
    idade["Paulao"] = 15;
    idade["Raul"] = 14;
    idade["Nicole"] = 13;
    idade["Leonardo"] = 15;
    idade["Cestari"] = 15;

    //Qual é a idade de Nicole?
    cout << idade["Nicole"] << endl;

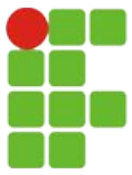
    return 0;
}
```



Vetores dinâmicos

STL

```
include <bits/stdc++.h>
```



Vetores dinâmicos

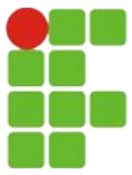
Um vetor é chamado de **dinâmico** quando não se tem a especificação do tamanho dele, ou seja, do número de elementos que ele contém.

Exemplo:

Armazenar os alunos do IFSP Barretos

Hoje pode ser 300 alunos

Amanhã pode ser 500 alunos



Vetores dinâmicos

Vetor **estático**: `int vetor[6];`

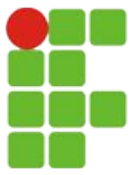
-5	12	2	0	0	0
----	----	---	---	---	---

vetor[0] vetor[1] vetor[2] vetor[3] vetor[4] vetor[5]

Vetor **dinâmico**: ????

-5	12	2	50	40	1	
----	----	---	----	----	---	-------

vetor[0] vetor[1] vetor[2] vetor[3] vetor[4] vetor[N]



Implementação

Declaração:

```
vector <int> vetor;
```

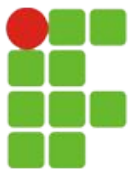
Armazenamento:

```
vetor.push_back(0);  
vetor.push_back(1);  
vetor.push_back(2);  
vetor.push_back(3);  
vetor.push_back(4);  
vetor.push_back(5);
```

vetor

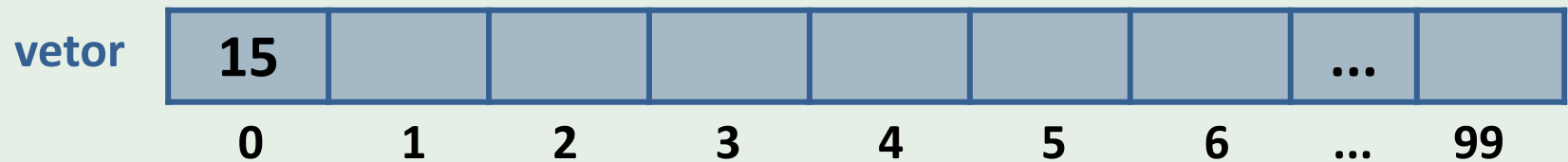
0	1	2	3	4	5
---	---	---	---	---	---

vetor[0] vetor[1] vetor[2] vetor[3] vetor[4] vetor[5]

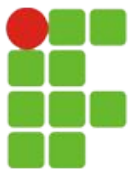


Implementação

Acessando o elemento do vetor:



```
cout << vetor[0];
```



Algumas funções importantes

size()

☐ retorna o nº de elementos do vetor

clear()

☐ remove todos os elementos do vetor

front()

☐ retorna o primeiro elemento do vetor

back()

☐ retorna o último elemento do vetor

sort()

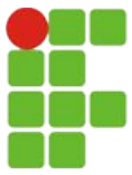
☐ ordena o vetor em ordem crescente ou decrescente.

pop_back()

☐ remove o último elemento do vetor

erase()

☐ remove um elemento específico do vetor



Algumas funções importantes

size()

□ retorna o nº de elementos do vetor

```
vector<int> vetor;
```

```
vetor.push_back(0);
```

```
vetor.push_back(1);
```

```
vetor.push_back(2);
```

```
vetor.push_back(3);
```

```
vetor.push_back(4);
```

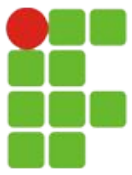
```
vetor.push_back(5);
```

```
int tamanho = vetor.size();
```

```
cout << tamanho << endl;
```

```
6
```

```
Process returned 0 (0x0)   execution time: 0.000 s  
Press any key to continue.
```



Algumas funções importantes

clear()

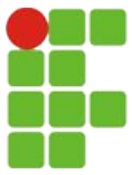
□ remove todos os elementos do vetor

```
vector<int> vetor;  
  
vetor.push_back(0);  
vetor.push_back(1);  
vetor.push_back(2);  
vetor.push_back(3);  
vetor.push_back(4);  
vetor.push_back(5);  
  
int tamanho = vetor.size();  
cout << tamanho << endl;  
  
vetor.clear();  
tamanho = vetor.size();  
cout << tamanho << endl;
```



```
6  
0
```

```
Process returned 0 (0x0)   exe  
Press any key to continue.
```



Algumas funções importantes

front()

□ retorna o primeiro elemento do vetor

back()

□ retorna o último elemento do vetor

```
vector<int> vetor;
```

```
vetor.push_back(0);
```

```
vetor.push_back(1);
```

```
vetor.push_back(2);
```

```
vetor.push_back(3);
```

```
vetor.push_back(4);
```

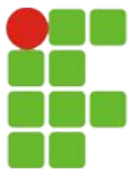
```
vetor.push_back(5);
```

```
cout << vetor.front() << endl;
```

```
cout << vetor.back() << endl;
```

```
0  
5
```

```
Process returned 0 (0x0)   e  
Press any key to continue.
```



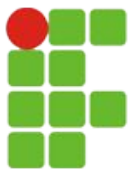
Algumas funções importantes

sort()

□ ordena o vetor em ordem crescente.

```
vector<int> vetor;  
  
vetor.push_back(5);  
vetor.push_back(4);  
vetor.push_back(1);  
vetor.push_back(3);  
vetor.push_back(2);  
vetor.push_back(0);  
  
//ordena em ordem crescente  
sort(vetor.begin(), vetor.end());  
for(int i=0; i < vetor.size(); i++){  
    cout << vetor[i] << " ";  
}
```

```
0 1 2 3 4 5  
Process returned 0 (0x0)   e  
Press any key to continue.
```



Algumas funções importantes

sort()

□ ordena o vetor em ordem decrescente.

```
vector<int> vetor;
```

```
vetor.push_back(5);
```

```
vetor.push_back(4);
```

```
vetor.push_back(1);
```

```
vetor.push_back(3);
```

```
vetor.push_back(2);
```

```
vetor.push_back(0);
```

```
//ordena em ordem decrescente
```

```
sort(vetor.begin(), vetor.end(), greater<>());
```

```
for(int i=0; i < vetor.size(); i++){
```

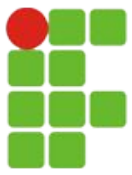
```
    cout << vetor[i] << " " ;
```

```
}
```

```
5 4 3 2 1 0
```

```
Process returned 0 (0x0)
```

```
Press any key to continue.
```

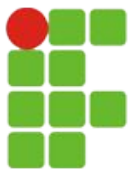


Algumas funções importantes

pop_back() □ remove o último elemento do vetor

```
vector<int> vetor;  
  
vetor.push_back(5);  
vetor.push_back(4);  
vetor.push_back(1);  
vetor.push_back(3);  
vetor.push_back(2);  
vetor.push_back(0);  
  
vetor.pop_back();  
  
for(int i=0; i < vetor.size(); i++){  
    cout << vetor[i] << " ";  
}
```

```
5 4 1 3 2  
Process returned 0 (0x0) ex  
Press any key to continue.
```

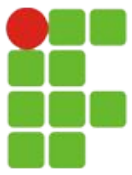


Algumas funções importantes

erase() □ remove um elemento do vetor, neste exemplo o primeiro elemento.

```
vector<int> vetor;  
  
vetor.push_back(5);  
vetor.push_back(4);  
vetor.push_back(1);  
vetor.push_back(3);  
vetor.push_back(2);  
vetor.push_back(0);  
  
// removing the first element  
vetor.erase(vetor.begin());  
  
for(int i=0; i < vetor.size(); i++){  
    cout << vetor[i] << " ";  
}
```

```
4 1 3 2 0  
Process returned 0 (0x0)  
Press any key to continue.  
_
```



- 1) Faça um programa que carregue um vetor com 7 números inteiros, calcule e mostre:
 - a) os números múltiplos de 2
 - b) os números múltiplos de 3
 - c) os números múltiplos de 2 e de 3
- 2) Faça um programa que receba a quantidade de peças vendidas por cada vendedor e armazene essas quantidades em um vetor. Receba também o preço da peça vendida de cada vendedor e armazene esses preços em outro vetor. Existem apenas 10 vendedores e cada vendedor pode vender apenas um tipo de peça, isto é, para cada vendedor existe apenas um preço. Calcule e mostre a quantidade total de peças vendidas por todos vendedores e para cada vendedor calcular e mostrar o valor total da venda, isto é, a quantidade de peças * o preço da peça.
- 3) Faça um programa que leia dois vetores (A e B) de 5 posições de números inteiros. O programa deve, então, subtrair o primeiro elemento de A do último de B, acumulando o valor, subtrair o segundo elemento de A do penúltimo de B, acumulando o valor, e assim por diante. Mostre o resultado da soma de todas as subtrações.
- 4) Faça um programa que receba o código e o preço de 5 produtos, calcule e mostre:
 - a) a quantidade de produtos com preço inferior a R\$ 50
 - b) o código dos produtos com preço entre R\$ 50 e R\$ 100
 - c) a média dos preços dos produtos com preço superior a R\$ 100.
- 5) Faça um programa que receba o total das vendas de cada vendedor e armazene-as em um vetor. Receba também o percentual de comissão de cada vendedor e armazene-os em outro vetor. Receba os códigos destes vendedores e armazene-os em um terceiro vetor. Existem apenas 10 vendedores. Calcule e mostre:
 - a) um relatório com os códigos dos vendedores e os valores a receber;
 - b) total das vendas de todos vendedores;
 - c) maior valor a receber e o código de quem o receberá;
 - d) menor valor a receber e o código de quem o receberá.
- 6) Faça um algoritmo que copie o conteúdo de um vetor em um segundo vetor.
- 7) Faça um algoritmo que faça a união de dois vetores de mesmo tamanho e mesmo tipo em um terceiro vetor com dobro do tamanho.